

Institute of Service Science, National Tsing Hua University
Service Oriented Architecture
Fall 2025

Duration: Sep 2025 – Dec 2025
Class Time: Mondays, 9am-12pm
Facebook Group: facebook.com/groups/ISS.SOAP

Instructor: Soumya Ray (soumya.ray@iss.nthu.edu.tw)
Assistant: 何捷睿 Jerry Ho (on MS Teams)

How do companies like Netflix, Airbnb, or ChatGPT continuously innovate their service? Innovation requires systems that we can confidently extend, redesign, retest, and redeploy. In this class, you will learn the best practices to develop the back-end systems that power scalable and innovative services.

This class is ideal for students with strong programming backgrounds who want to one day be:

Senior developers: develop scalable, extensible applications in startups and large enterprises

Tech leads: design and architect complex IT solutions, lead and mentor other developers

IT product managers: manage sophisticated IT teams, understand complex IT solutions

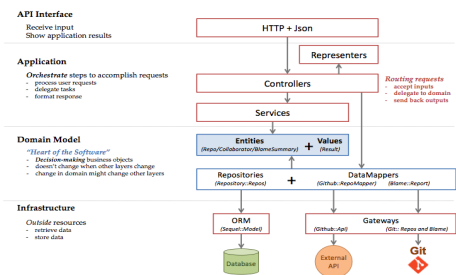
What techniques will I learn?

Learn architectural and refactoring techniques to create systems that are easy to maintain and extend:

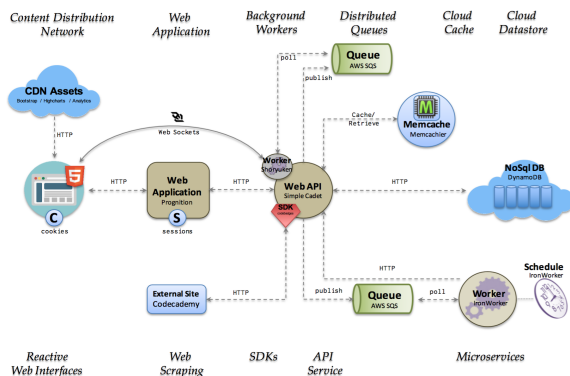
Domain Driven Design: layered architecture that separates business logic from infrastructure and presentation

Continuous Integration: automated unit & integration tests in to ensure correct system behavior on multiple platforms

Functional flow: monads, transactions, railway-oriented programming to explicitly manage errors



What tools will we use?



Cloud Services: Amazon AWS, RedisCloud, Heroku

Distributed Systems: Web APIs, Apps, Microservices

Data Stores: Relational databases & key-value stores; local, hosted, and cloud-based solutions

Reactive UX: background jobs, message queues, web sockets, reactive front-end elements

Responsive UI: mobile-first, responsive web-interfaces

DevOps: Continuous development and integration

Version Control: git, Github Flow, rebase, pull requests

How much coding should I already know?

You should be able to create a basic application using:

a) *Object-oriented programming:* any OOP language such as Javascript, Python, PHP, Java, Ruby, C#, C++, etc.

b) *Basic web page design:* HTML + CSS

c) *Database design:* Relational design + ERD + SQL

This class will use the Ruby programming language, but the techniques are relevant to any language

We will use services like Postgres, AWS, and Heroku but the skills you learn apply to any tech stack

The platforms and tools you will use are free up to the requirements of small businesses

How will I be graded?

Individual Development Effort: 80% (from your **code contributions** on Github and other HW)
Attendance + Online Discussion: 20% (you **must share your thoughts** in-class or MS Teams)

INTRODUCTION

Service Orientation

Service Providers
Service Architecture
Meet Your Tools

Coding Style and Conventions

Programming Tools
Idiomatic Coding
“Good” Code

Architecture Matters

IT systems become slow and as they grow in complexity. How are leading IT companies managing their code? Let's learn how to improve innovation and bring joy.

Writing Confident Code

We are all afraid to show our code. How can we become confident at coding and be proud of what we make? Let's start by following the best practices of clean coding.

BACK TO BASICS

Refactoring

Test, Code, Refactor Cycle
Code Quality Metrics
Code Complexity

Quantifying Code Quality

How should we structure to solve small problems? Let's quantify what it means to have good code using metrics.

Object-Orientation

Procedural Code
Objects and Message Passing
Composition, Metaprogramming, and more

Capturing objects

Why is object orientation so prevalent in business programming? We will dig deep into the philosophy of OOP and how to use it elegantly.

(evening workshop: Git/Gitflow)

Git Version Control Workshop

Web APIs

Serialization/Deserialization
API Exploration
API Library

Getting down with data

We will develop our first library to extract, transform, and load data from interesting APIs.

Testing and Tooling

Testing Tools
Library Tools
UPSTART: Elevator Pitch

Fast and Reliable Tests

Testing applications with external resources is unreliable and inconvenient. Let's use stubs to capture and replay data flows. And let's upgrade our code management tools.

LAYERED ARCHITECTURE

Application Architecture

Challenges of Enterprise Architecture
Enterprise Design Patterns
MVC Architecture

Layered architecture

Even a simple modular architecture will make your code clean, will make you more confident to change your code, and will let you sleep better at night.

Database Infrastructure

Databases and Infrastructure
Is MVC Enough?
The Repository Pattern

Layered architecture

Even a simple modular architecture will make your code clean, will make you more confident to change your code, and will let you sleep better at night.

Domain Driven Design

Domain Complexity
Domain Modeling
Cloud Deployment

What's your value proposition?

The value of our software comes from its domain logic. Let's make our code match our business language.

Presentation Layer

Sessions and Error Checking
Acceptance Testing
Refactoring Views

Real artists ship!

Let's see how to design and build an interface responsibly. And then let's deploy our system early and often using continuous integration, delivery, and deployment.

Application Layer

Form Objects
Service Objects
Monadic Chains

Managing our Transactions

Our controllers are getting fat! We will see how to encapsulate and test major use cases of the user experience.

DISTRIBUTED ARCHITECTURE

API-centric Architecture

Creating a Web API
Representational State Transfer - REST
Layered API Architecture

Growing Pains

Our systems must be able to scale up and also respond to many types of clients: web applications, mobile apps, etc. Can we split our system into two without major headaches?

Caching and Performance

Network Latency
Proxy Caching
Reverse-Proxy Caching

Multitasking 1

All this network input/output is making our service slow. We've got to learn to multitask: let's start new things while waiting for machines to respond.

Asynchronicity

Concurrency
Parallelism
Background Workers

Multitasking

Some tasks need a long time to finish. Can we start serving new users while still processing old tasks? Let's use concurrency, queues and workers to handle all our tasks.

Reactive Architecture

Reactive Architecture
Web Sockets
PubSub Pattern

Show me some progress

Users hate waiting. Let's give them a real-time peek at what our system is doing. But HTTP is sometimes not enough: time to bring out websockets.

PRODUCT LAUNCH

Show and Tell

UPSTART: Presentations
Why Architecture Matters
What's Next

It's show time!

*Let's see what amazing things we have made.
See how confident and proud we can be of our code!
And let's talk about what we will accomplish in future.*

Artificial Intelligence (AI) Use Policy

Grounded in the principles of transparency and responsibility, this course encourages students to leverage Generative AI (GenAI) like ChatGPT and Github CoPilot, and other coding resources like StackOverflow to enhance their learning and improve the quality of their course outputs.

This means students may use GenAI or other resources with no restrictions or disclosure.

However, we expect students to carefully study, verify, and suitably alter their GenAI assisted solutions to match requirements and reflect their genuine learning.